# Parallel Matrix Multiplication Analysis (Part 2 – Exercises 1 and 2)

## Index

1. Parallel Matrix Multiplication Analysis (Part 2 – Exercises 1 and 2)
2. Introduction
3. Experimental Setup
4. Metrics Overview
5. Exercise 1 – Parallel Strategy Comparison 5.1 Execution Time 5.2 GFLOPS 5.3 Speedup 5.4 IPC 5.5 Short Cache Note 5.6 Strategy Comparison
6. Exercise 2 – Thread Scaling Analysis 6.1 Execution Time vs Threads 6.2 Speedup vs Threads 6.3 Efficiency vs Threads 6.4 GFLOPS vs Threads 6.5 IPC vs Threads 6.6 Advanced Scaling Insights 6.7 Strategy Comparison
7. Discussion
8. Conclusion

## 1. Introduction

In this part of the project, the goal is not only to see which line is higher or lower in the plots, but to explain why that happens for the actual implementations we wrote. Exercise 1 compares OnMult - omp parallel for, OnMult - omp parallel + omp for, and OnMultLine - parallel with a fixed thread count. Exercise 2 uses OnMultLine with omp parallel for, omp parallel for collapse(2), and omp for simd while increasing the number of threads.

The main idea across all sections is to connect performance to OpenMP behavior in the code. That means looking at how threads are created, how loop iterations are split, when synchronization happens, and how much time is spent doing useful matrix multiplication versus runtime overhead.

## 2. Experimental Setup

Exercise 1 was executed with 4 threads and matrix sizes from 1024 to 3072. This setup is useful because it keeps thread count constant and makes it easier to isolate differences caused by the OpenMP constructs in OnMult and OnMultLine.

Exercise 2 was executed with fixed matrix size 8192 and threads from 4 to 24. This setup is focused on scaling behavior of OnMultLine with omp parallel for, omp parallel for collapse(2), and omp for simd.

All results come from perf outputs, including execution times and hardware counters. The interpretation below is based on those measurements and on how the OpenMP directives are used in each implementation.

## 3. Metrics Overview

Execution time shows whether a given implementation actually reduces wall-clock runtime.

GFLOPS shows how much useful floating-point work is sustained per second.

Speedup compares a threaded run with the 4-thread baseline used in each Exercise 2 method.

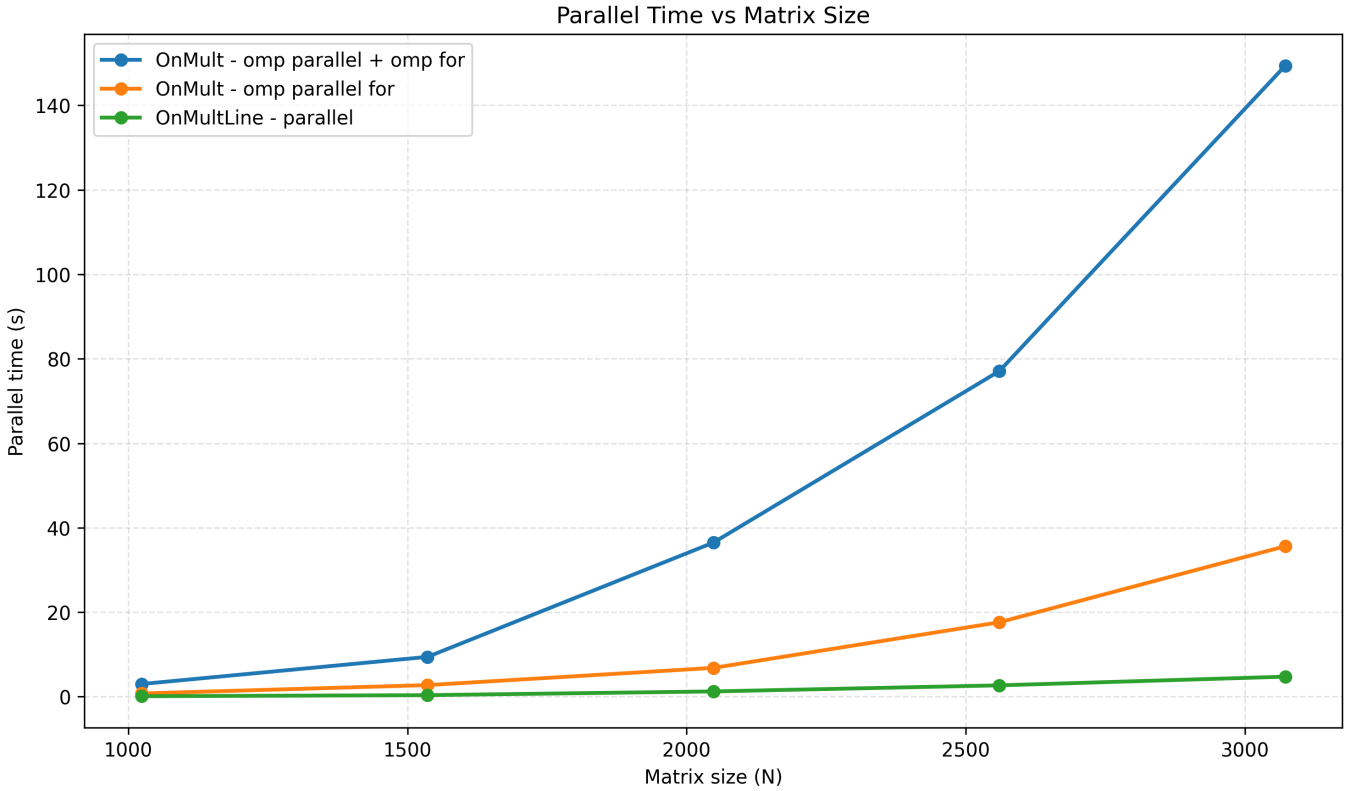
Efficiency shows how much useful speedup each added thread still contributes.

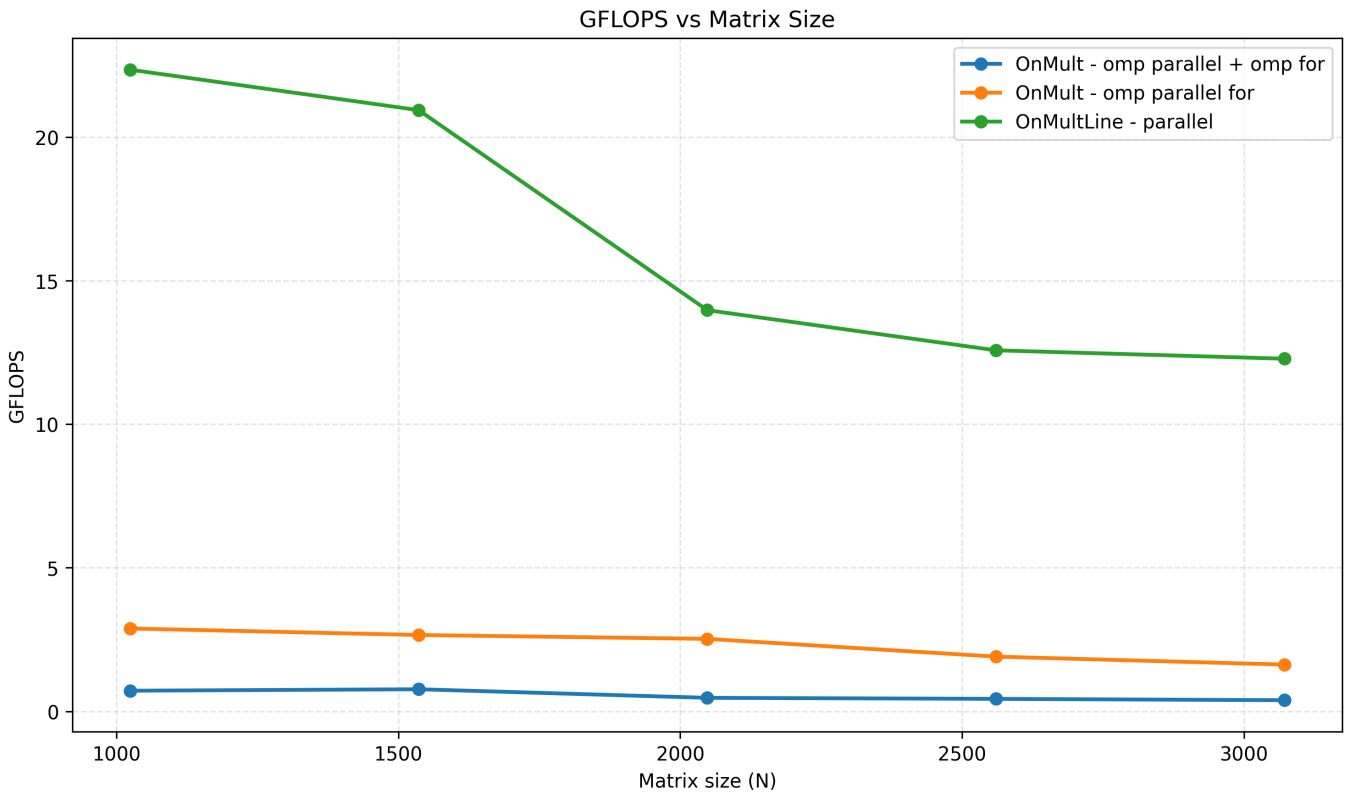
IPC shows how effectively CPU cycles are converted into retired instructions.

Cache miss rate gives secondary context when we need to explain scaling changes.

# 4. Exercise 1 – Parallel Strategy Comparison

## 4.1 Execution Time





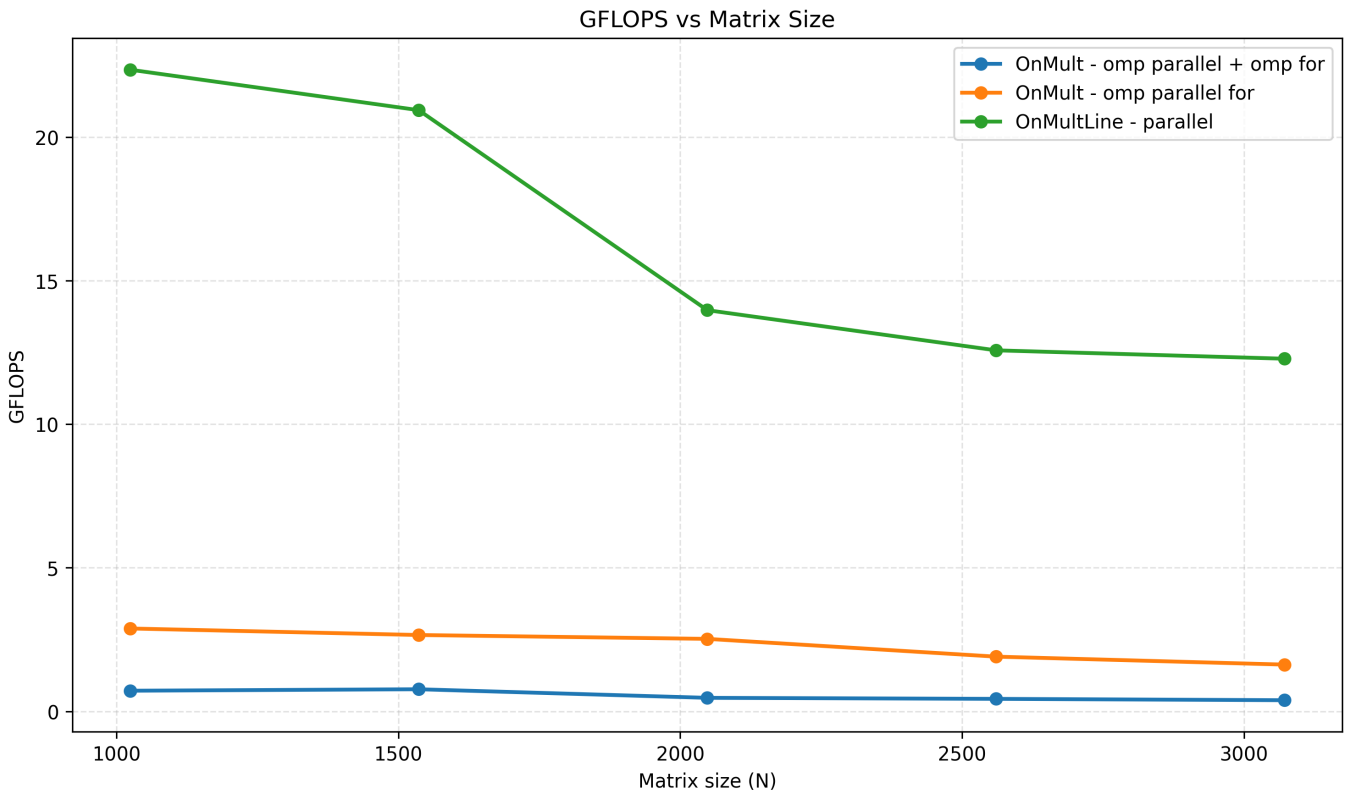

The results show a consistent ordering across all matrix sizes: **OnMultLine - parallel** is the fastest implementation, followed by **OnMult - omp parallel for**, while **OnMult - omp parallel + omp for** performs significantly worse. The GFLOPS plot confirms the same ranking, indicating that the differences in execution time directly translate into differences in computational throughput.

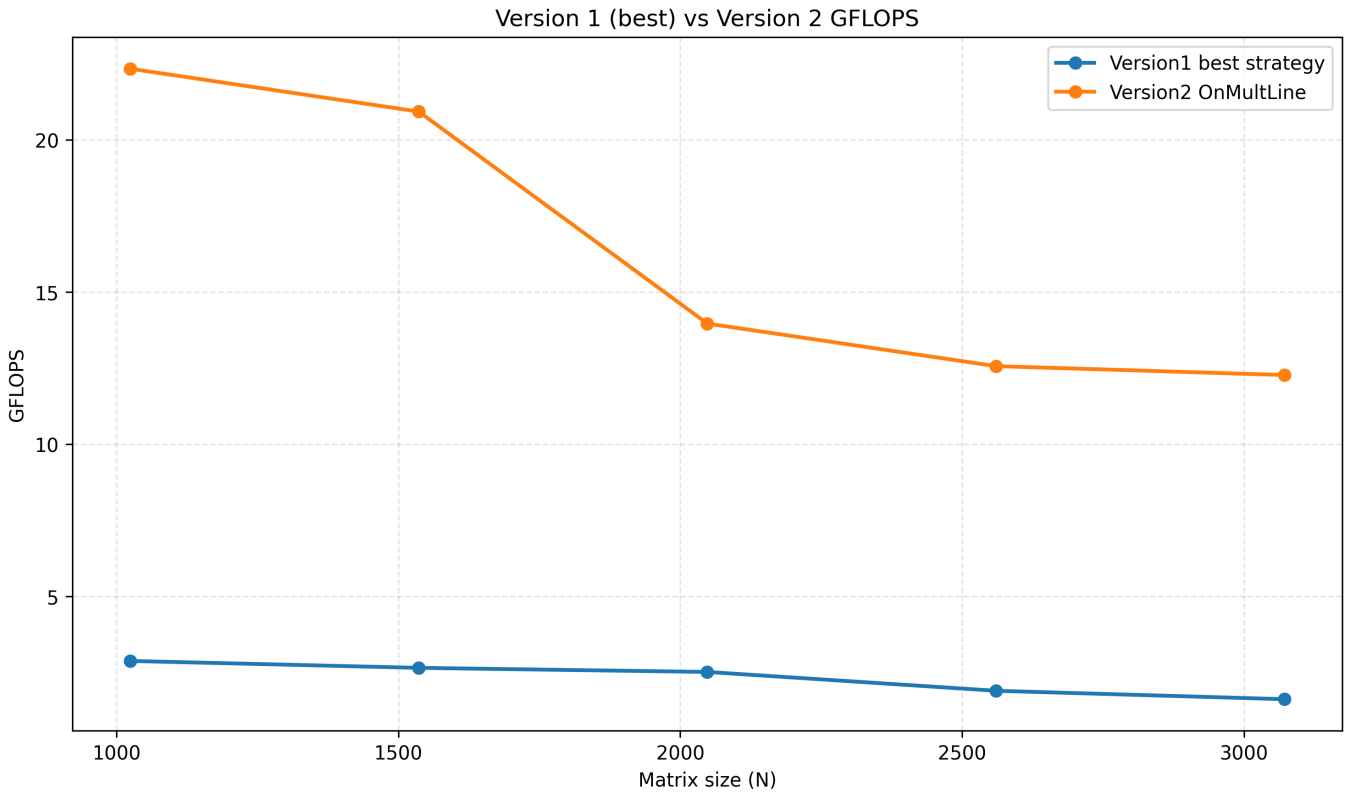
This behavior is explained by how each implementation uses OpenMP. In **OnMult - omp parallel for**, thread creation and loop distribution are handled in a single directive, minimizing runtime overhead and allowing threads to move quickly into useful computation. In contrast, **OnMult - omp parallel + omp for** separates the parallel region from the work-sharing construct, introducing additional synchronization and coordination costs that reduce the effective work done by each thread. These overheads become more significant as the matrix size increases, leading to poorer scalability.

**OnMultLine - parallel** achieves the best performance because it maintains a more balanced and continuous distribution of work across threads, ensuring that threads remain active for a larger portion of execution time. This results in both lower execution time and higher GFLOPS.

Overall, the results show that with a fixed number of threads, performance depends strongly on how efficiently the OpenMP directives are structured. Implementations that minimize overhead and keep threads focused on useful work achieve better performance, while those with additional synchronization costs scale poorly.

## 4.2 GFLOPS




The GFLOPS results show a clear and consistent ordering: **OnMultLine - parallel** achieves the highest throughput across all matrix sizes, followed by **OnMult - omp parallel for**, while **OnMult - omp parallel + omp for** remains significantly lower. The direct comparison between Version 1 and Version 2 confirms that this gap is consistent and not dependent on a specific matrix size.

This behavior is explained by how effectively each implementation converts execution time into useful floating-point work. In **OnMult - omp parallel for**, the combined directive allows threads to enter the computation phase with minimal coordination overhead, resulting in moderate but stable throughput. In

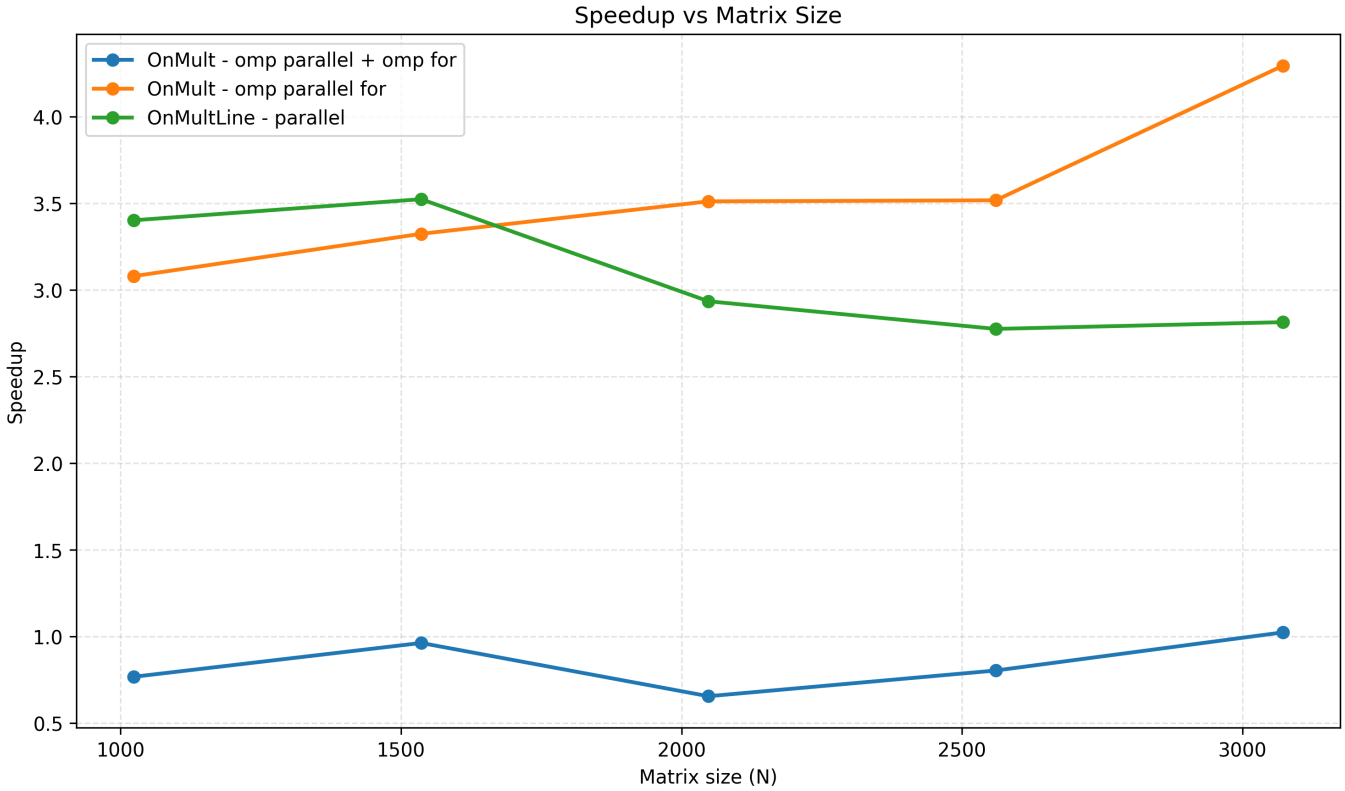
contrast, **OnMult - omp parallel + omp for** introduces additional synchronization and runtime management due to the separation of the parallel region and the loop, reducing the fraction of time spent performing arithmetic operations and therefore lowering GFLOPS.

**OnMultLine - parallel** achieves the highest GFLOPS because it maintains a higher density of useful work within each thread. The loop structure and parallelization allow threads to remain active with fewer interruptions, which leads to better utilization of the available processing capacity. As a result, it consistently outperforms both Version 1 implementations in terms of computational throughput.

A slight decrease in GFLOPS is observed for all implementations as matrix size increases, indicating reduced parallel efficiency for larger workloads. However, the relative differences between implementations remain stable, showing that the dominant factor is the efficiency of the OpenMP structure rather than the problem size itself.

Overall, the GFLOPS results reinforce that performance depends on how directly the OpenMP directives map threads to useful computation. Implementations that minimize coordination overhead and keep threads focused on arithmetic operations achieve higher throughput, while those with additional synchronization costs perform significantly worse.

### 4.3 Speedup




The speedup results show a clear distinction between the implementations. **OnMult - omp parallel for** achieves the highest speedup, approaching and even slightly exceeding the ideal value for 4 threads, while **OnMultLine - parallel** maintains a stable but lower speedup. In contrast, **OnMult - omp parallel + omp for** performs poorly, with speedup remaining close to 1, indicating that parallelization brings little benefit in this case.

This behavior is directly related to how efficiently each implementation uses the available threads. In **OnMult - omp parallel for**, the OpenMP directive combines thread creation and loop distribution, allowing

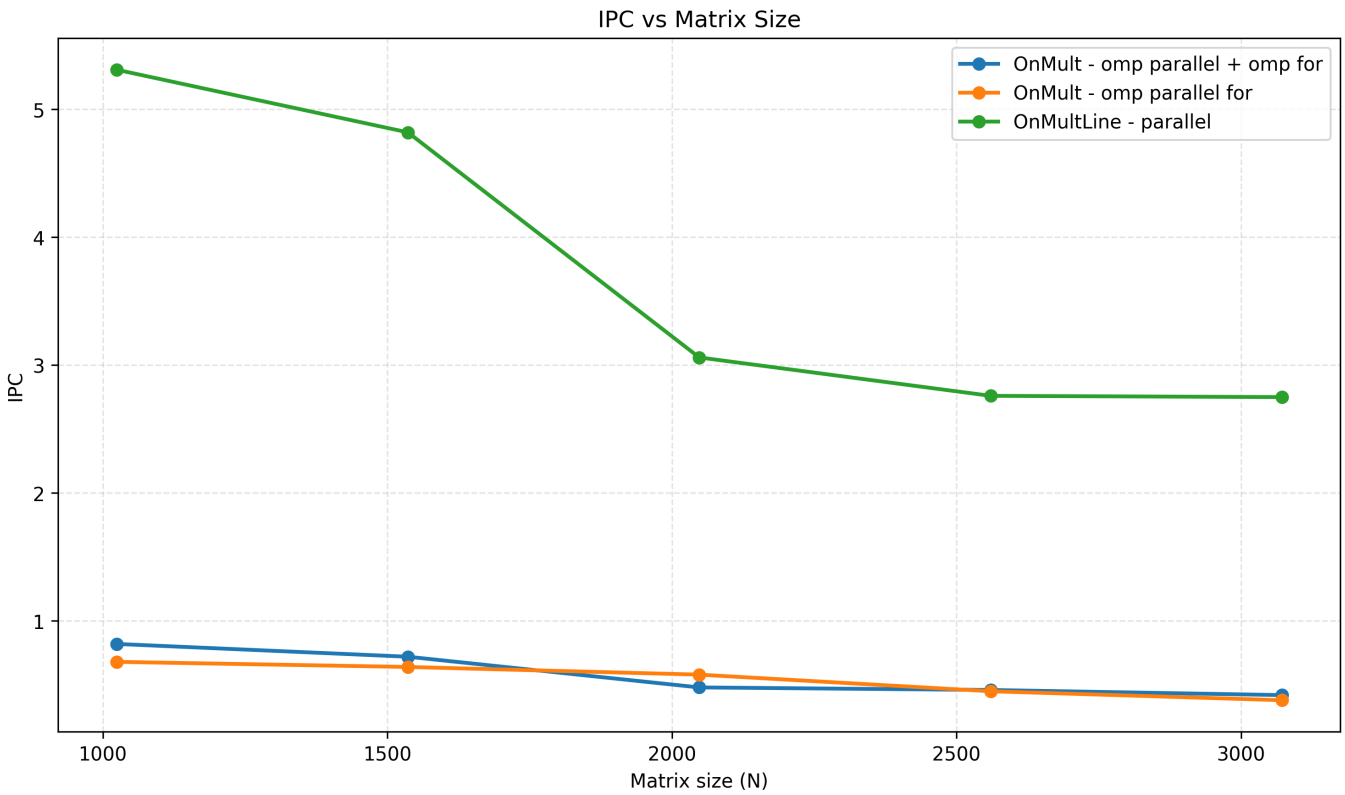
work to be assigned to threads with minimal overhead. This results in a higher fraction of execution time being spent on actual computation, which translates into better speedup. In **OnMult - omp parallel + omp for**, the separation between the parallel region and the work-sharing construct introduces additional synchronization and coordination costs. These overheads reduce the effective contribution of each thread, so the theoretical speedup is not achieved in practice.

Although **OnMultLine - parallel** is the fastest implementation in absolute terms, its speedup is lower because its sequential version is already more efficient, leaving less room for improvement through parallelization. As a result, even though it performs more work per unit time, the relative gain compared to its sequential baseline is smaller.

Overall, the results show that speedup is not determined only by the number of threads, but by how effectively those threads are used. Implementations that minimize overhead and keep threads focused on useful work, such as **omp parallel for**, achieve better scaling, while inefficient parallel structures can significantly limit the benefits of parallel execution.

**Note:** Efficiency is not shown separately because, with a fixed number of 4 threads, it would simply be the speedup graph scaled by a constant factor.

### 4.4 IPC




The IPC results follow the same pattern observed in execution time and GFLOPS: **OnMultLine - parallel** achieves the highest IPC across all matrix sizes, **OnMult - omp parallel for** remains moderate, and **OnMult - omp parallel + omp for** consistently shows the lowest values. This indicates that the implementations differ significantly in how effectively they use CPU cycles.

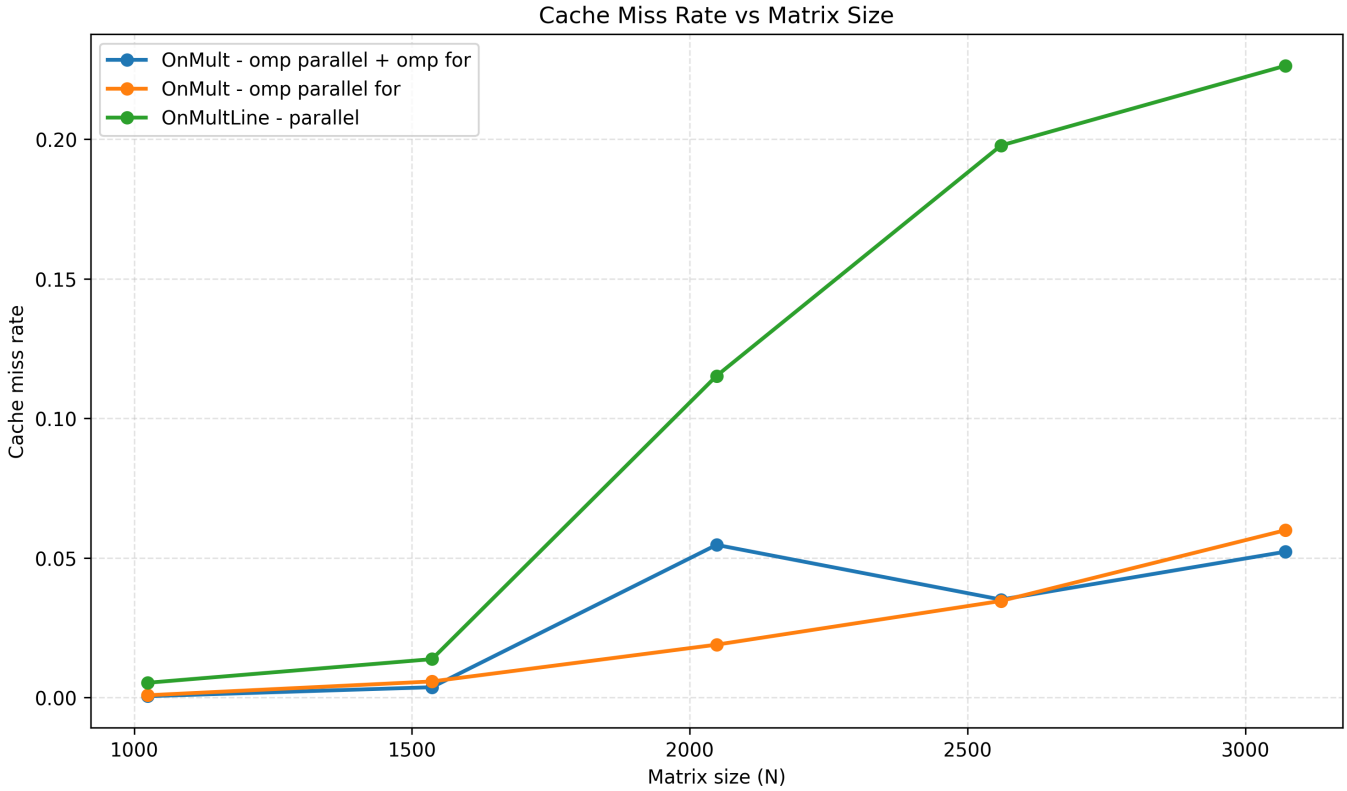
This behavior is explained by how much time threads spend performing useful computation versus handling OpenMP overhead. In **OnMult - omp parallel for**, the combined directive allows threads to quickly enter the computation phase and remain there, resulting in a reasonable IPC. In contrast, **OnMult - omp parallel +**

**omp for** introduces additional synchronization and runtime coordination, causing threads to spend more cycles outside arithmetic work, which lowers IPC. **OnMultLine - parallel** achieves the highest IPC because its parallel structure keeps threads continuously active with fewer interruptions, allowing instructions to be executed more consistently.

As matrix size increases, IPC decreases for all implementations, indicating reduced execution efficiency as the workload grows. However, the relative ordering remains stable, showing that the main factor is not the workload itself, but how effectively each implementation keeps the CPU focused on useful work.

Overall, IPC confirms that implementations with lower parallel overhead and better thread utilization execute instructions more efficiently, directly contributing to better performance.

### 4.5 Short Cache Note

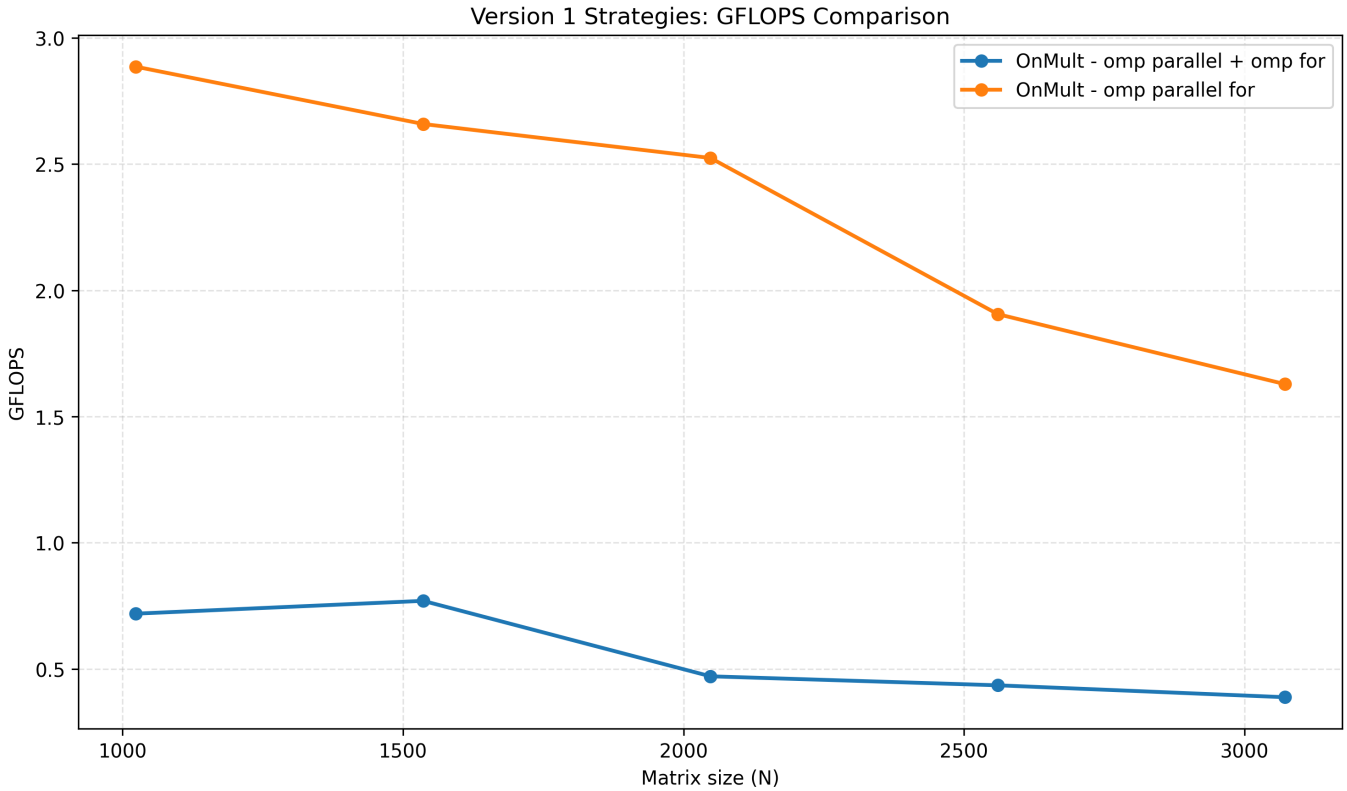



The cache miss rate increases with matrix size for all implementations, with **OnMultLine - parallel** showing the highest values, while both Version 1 implementations remain significantly lower. However, this trend does not match the performance results, since **OnMultLine - parallel** is still the fastest implementation despite having the highest miss rate.

This shows that, in this exercise, cache behavior alone does not explain performance differences. With a fixed number of threads, the dominant factor is how efficiently each OpenMP construct manages parallel execution. In particular, **OnMult - omp parallel + omp for** performs worse even with relatively moderate miss rates, indicating that its main limitation comes from additional synchronization and runtime overhead rather than cache inefficiency.

Overall, the cache results provide useful context but are not the main driver of performance in Exercise 1. The differences between implementations are better explained by how effectively each approach minimizes overhead and keeps threads performing useful work.

### 4.6 Strategy Comparison



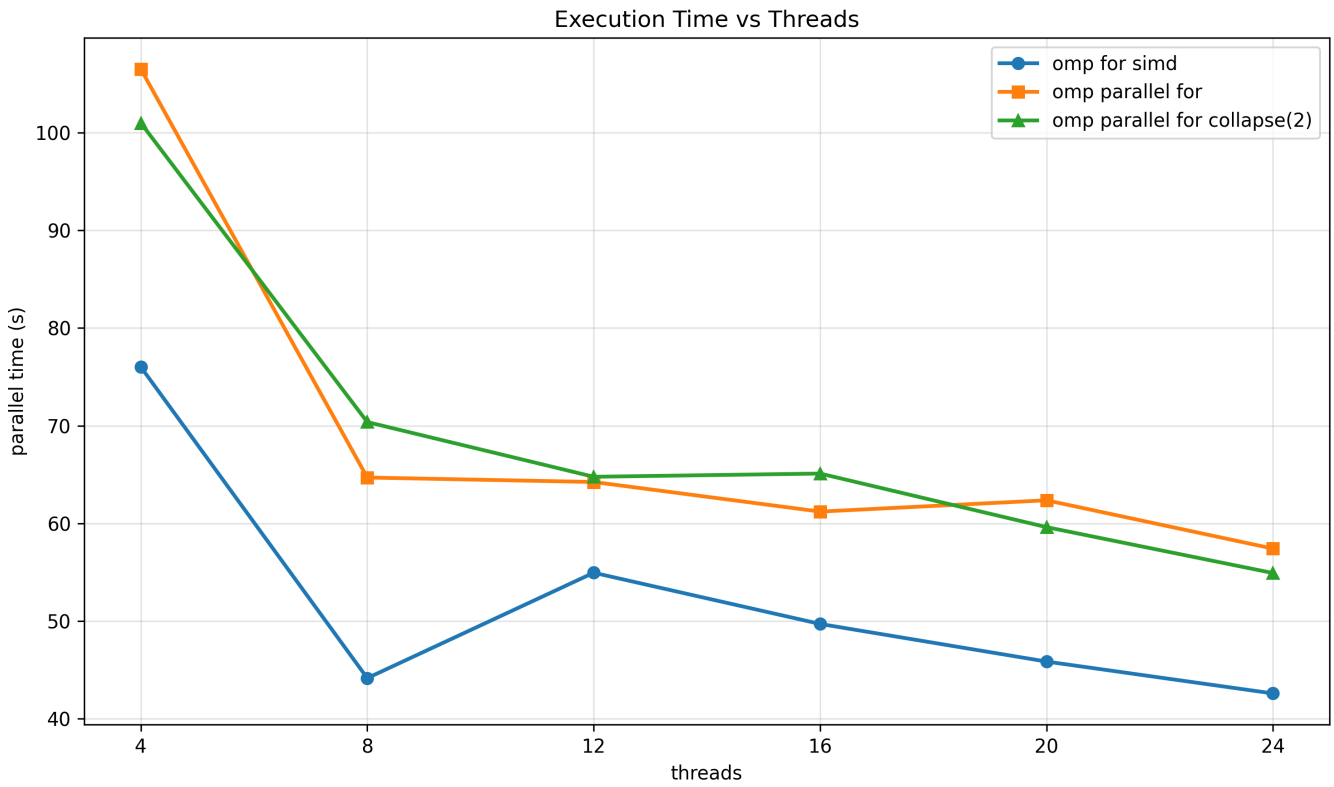

The comparison between the two Version 1 strategies shows that **OnMult - omp parallel for** consistently achieves higher GFLOPS than **OnMult - omp parallel + omp for** across all matrix sizes, confirming that the difference in performance is systematic rather than incidental.

This behavior is directly explained by the OpenMP structure. In **OnMult - omp parallel for**, thread creation and loop distribution are handled in a single directive, allowing iterations to be assigned with minimal runtime overhead and keeping threads focused on useful computation. In contrast, **OnMult - omp parallel + omp for** separates the parallel region from the work-sharing construct, introducing additional synchronization and coordination costs. These extra steps do not contribute to computation and reduce the effective work performed by each thread, leading to lower throughput.

Overall, this comparison shows that, for this kernel, the choice of OpenMP construct has a direct impact on performance: a more compact directive structure (**omp parallel for**) results in better efficiency, while splitting the parallelization into multiple constructs introduces overhead that significantly limits performance.

## 5. Exercise 2 – Thread Scaling Analysis

### 5.1 Execution Time vs Threads

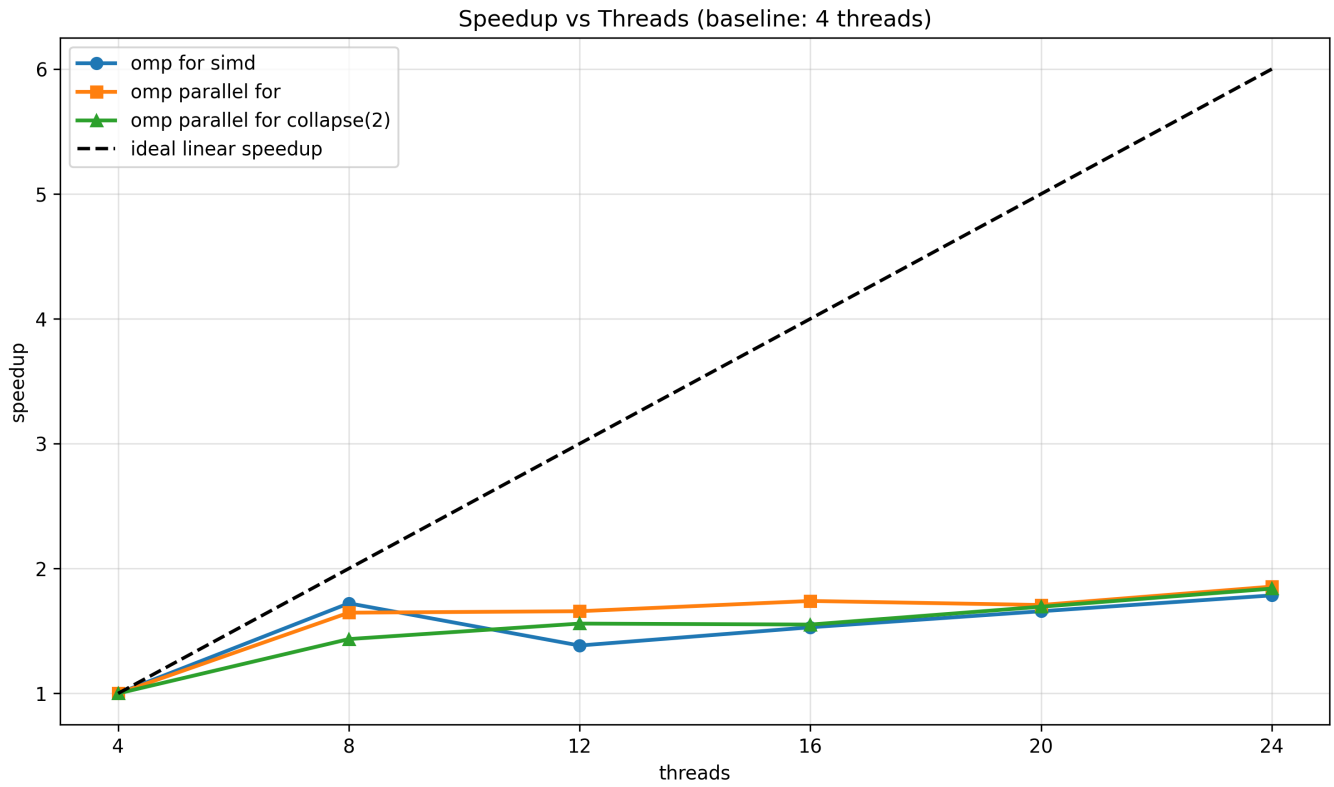



Execution time decreases significantly when moving from 4 to 8 threads for all implementations, but the improvement slows down as the number of threads increases, showing clear diminishing returns. Across all thread counts, **omp for simd** consistently achieves the lowest execution time, followed by **omp parallel for**, while **omp parallel for collapse(2)** starts slower but becomes competitive at higher thread counts.

This behavior is explained by how each strategy scales with additional threads. At low thread counts, adding threads increases the amount of work done in parallel with relatively small overhead, leading to strong improvements. As the number of threads grows, however, synchronization, scheduling, and resource contention become more significant, reducing the benefit of each additional thread. In **omp parallel for collapse(2)**, flattening the iteration space improves load distribution, which helps at higher thread counts, but it also increases scheduling overhead. In **omp for simd**, vectorization improves per-thread computation, allowing it to maintain the best execution time, but it still suffers from the same global overheads as thread count increases.

Overall, the results show that all implementations benefit from parallelization, but only up to a certain point. Beyond that, increasing the number of threads leads to smaller gains because overhead and contention begin to limit scalability.

## 5.2 Speedup vs Threads

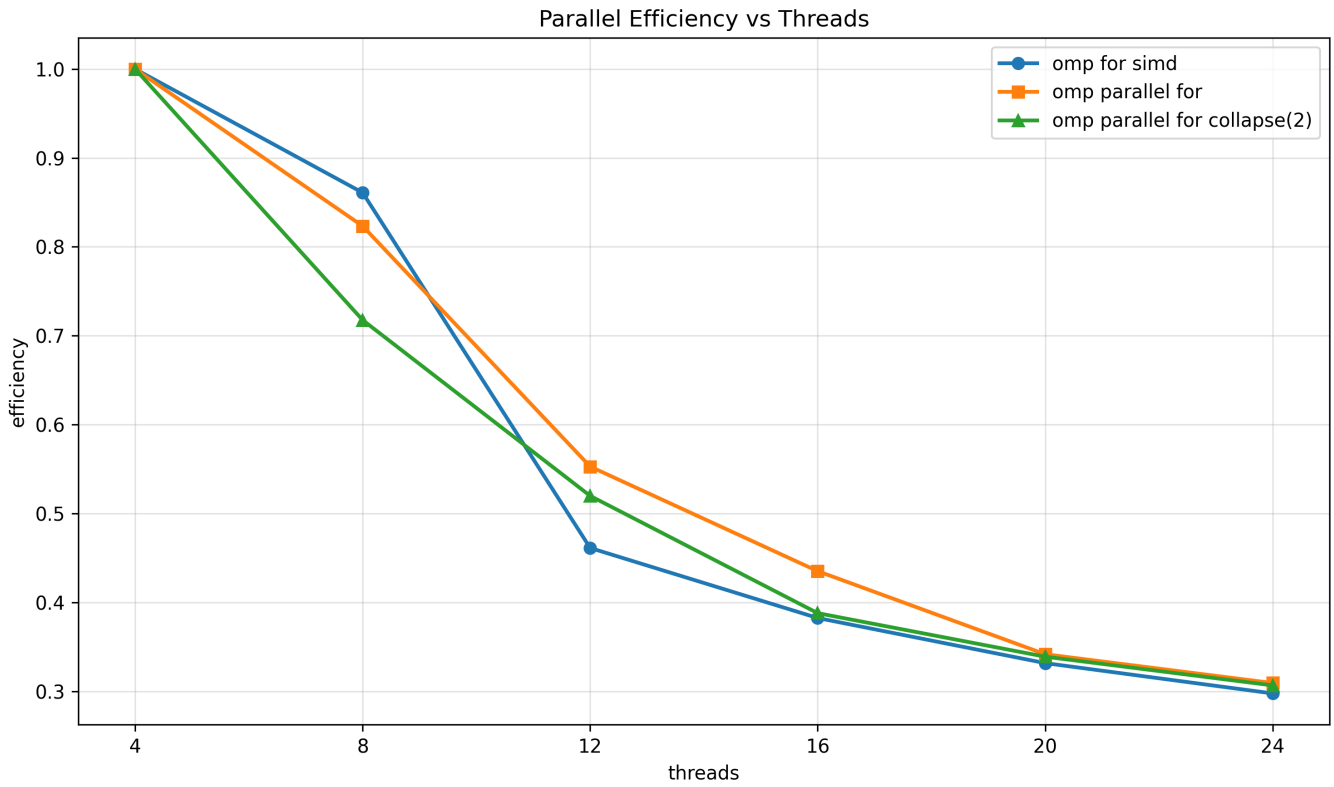



The speedup results are clearly sub-linear for all implementations, with all curves diverging significantly from the ideal linear scaling line as the number of threads increases. **omp parallel for** and **omp for simd** achieve slightly higher speedup overall, while **omp parallel for collapse(2)** starts lower but gradually converges as thread count increases.

This behavior is explained by the increasing impact of parallel overhead and resource contention. At low thread counts, adding threads directly increases useful parallel work, leading to noticeable speedup gains. However, as more threads are added, the cost of synchronization, scheduling, and coordination grows, reducing the effectiveness of each additional thread. In **omp parallel for collapse(2)**, flattening the iteration space improves load balancing, which helps recover performance at higher thread counts, but it also introduces additional scheduling overhead. In **omp for simd**, vectorization improves per-thread efficiency, but it does not eliminate the overhead associated with managing a larger thread team.

As a result, each additional thread contributes less to overall speedup, leading to the observed saturation. This shows that scalability is limited not by the amount of parallel work alone, but by how efficiently threads can be coordinated and kept productive as their number increases.

### 5.3 Efficiency vs Threads

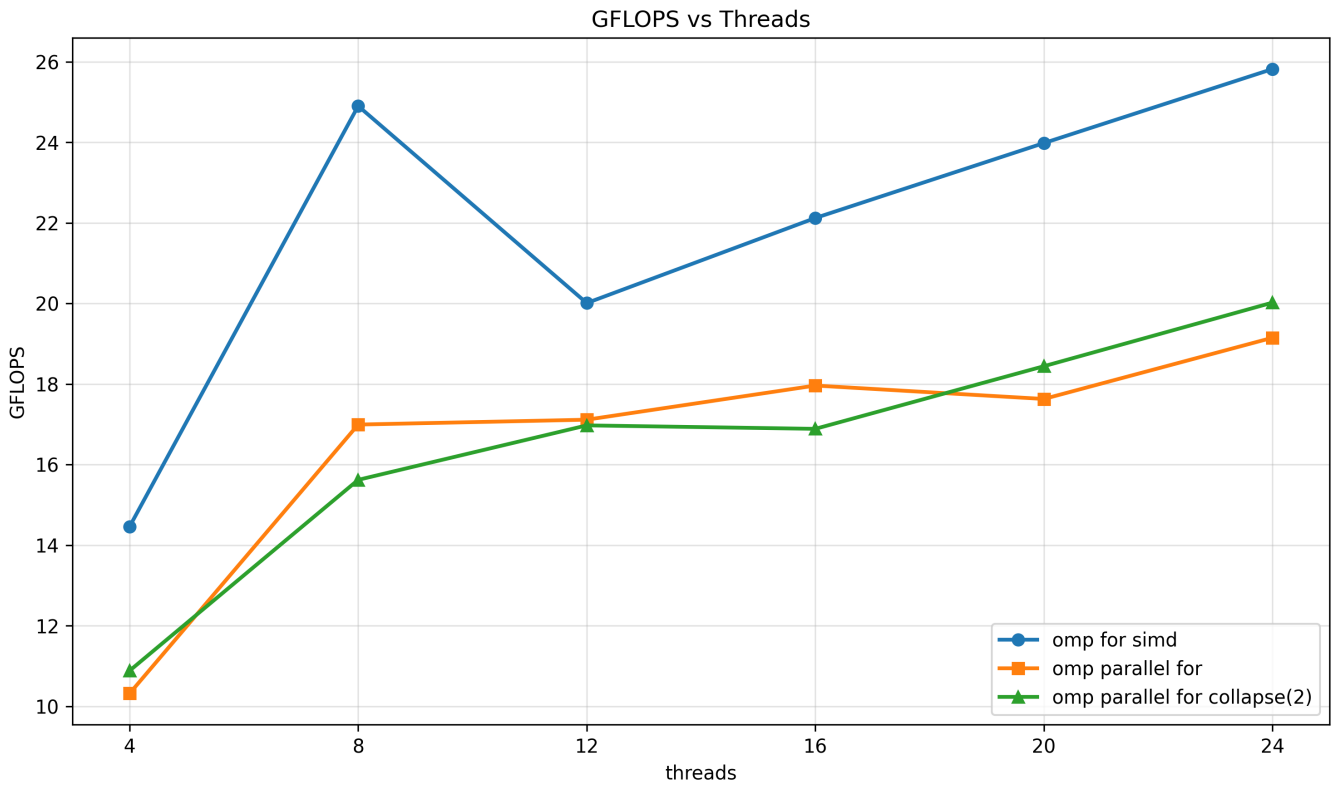


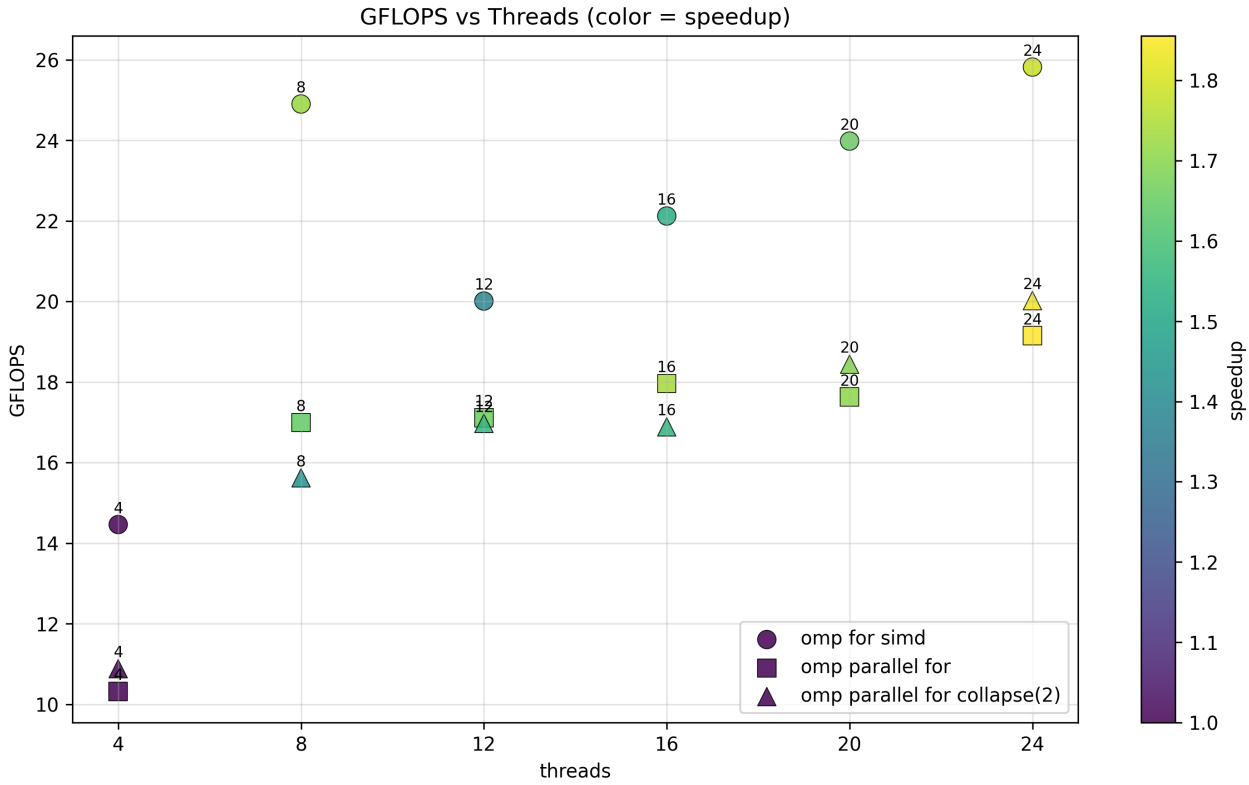

Efficiency decreases for all implementations as the number of threads increases, even though execution time continues to improve. **omp parallel for** maintains the highest efficiency across most thread counts, while **omp parallel for collapse(2)** starts lower but follows a similar trend, and **omp for simd** drops more sharply after the initial gains.

This behavior is caused by the growing impact of parallel overhead as more threads are introduced. In **omp parallel for** and **omp parallel for collapse(2)**, increasing the number of threads raises synchronization and scheduling costs, reducing the amount of useful work performed per thread. Although **collapse(2)** improves load distribution by expanding the iteration space, it also adds scheduling overhead that limits its efficiency. In **omp for simd**, vectorization improves computation within each thread, but it does not reduce the overhead associated with managing a larger thread team, leading to a faster drop in efficiency.

Overall, the results show that scalability is limited by how quickly efficiency decreases. The most effective implementation is not the one that uses the most threads, but the one that maintains a higher fraction of useful work as thread count increases.

### 5.4 GFLOPS vs Threads



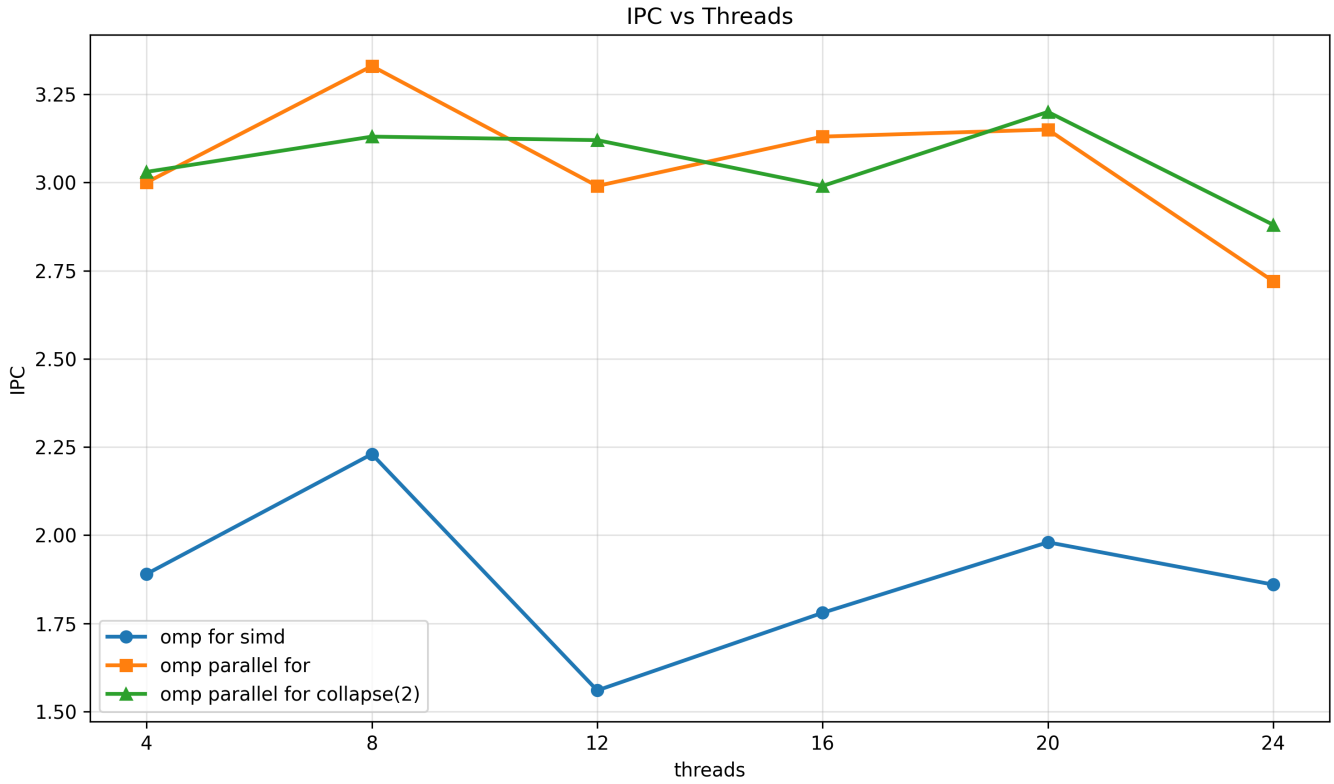

GFLOPS increases significantly from 4 to 8 threads for all implementations, then grows more slowly and begins to plateau at higher thread counts. **omp for simd** achieves the highest throughput across all configurations, followed by **omp parallel for**, while **omp parallel for collapse(2)** improves steadily and becomes competitive at higher thread counts. The scatter plot confirms that higher thread counts do not always translate into proportional throughput gains.

This behavior is explained by the balance between useful computation and parallel overhead. At low thread counts, adding threads increases the amount of work executed in parallel, leading to strong gains in GFLOPS.

As thread count grows, synchronization, scheduling, and resource contention limit how effectively additional threads can be used. In **omp parallel for**, loop distribution is efficient, but coordination costs increase with more threads. In **omp parallel for collapse(2)**, expanding the iteration space improves load balancing, which helps at higher thread counts, but the added scheduling complexity reduces efficiency. **omp for simd** achieves the highest GFLOPS because vectorization increases the amount of work performed per thread, but it still experiences the same plateau once thread-level overhead dominates.

Overall, the results show that higher throughput depends on both per-thread efficiency and overall thread coordination. While **omp for simd** maximizes computation within each thread, all implementations eventually reach a point where parallel overhead limits further gains.

### 5.5 IPC vs Threads

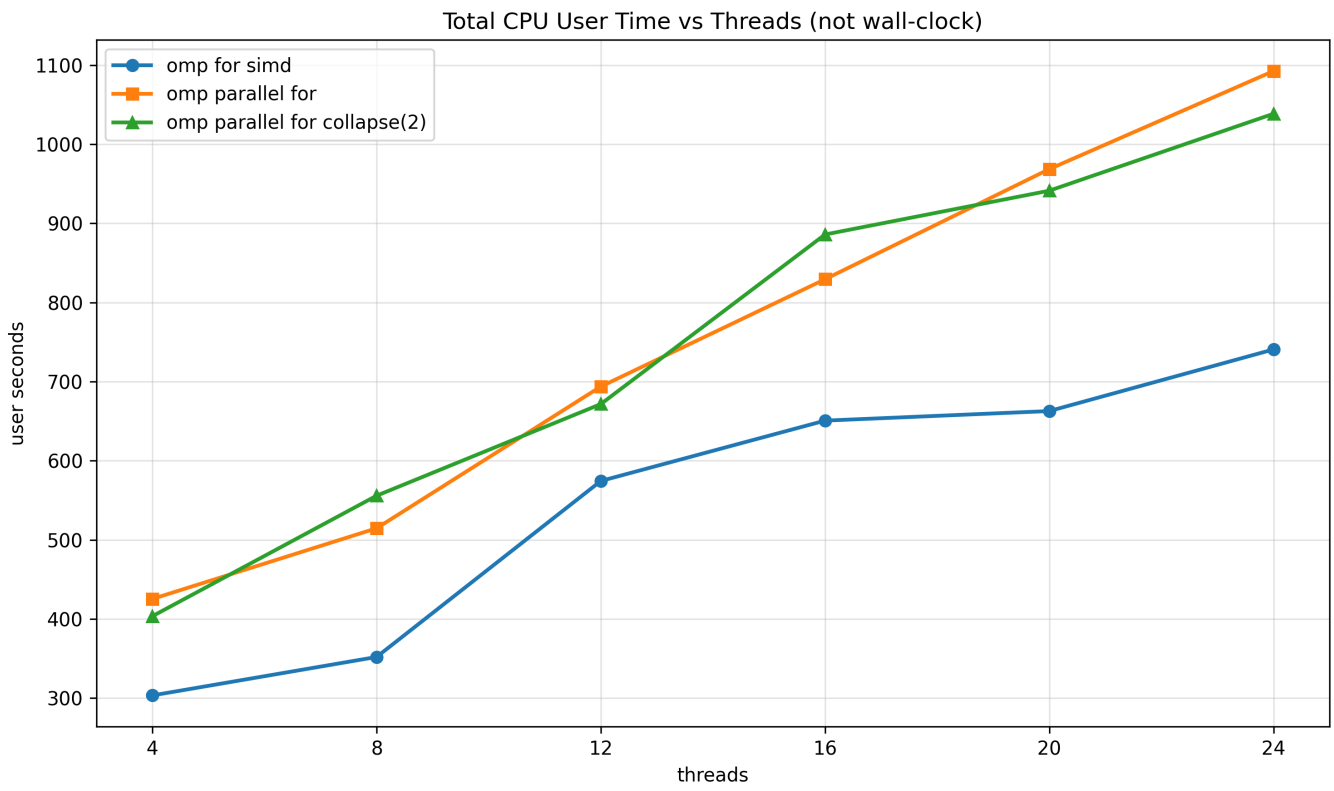


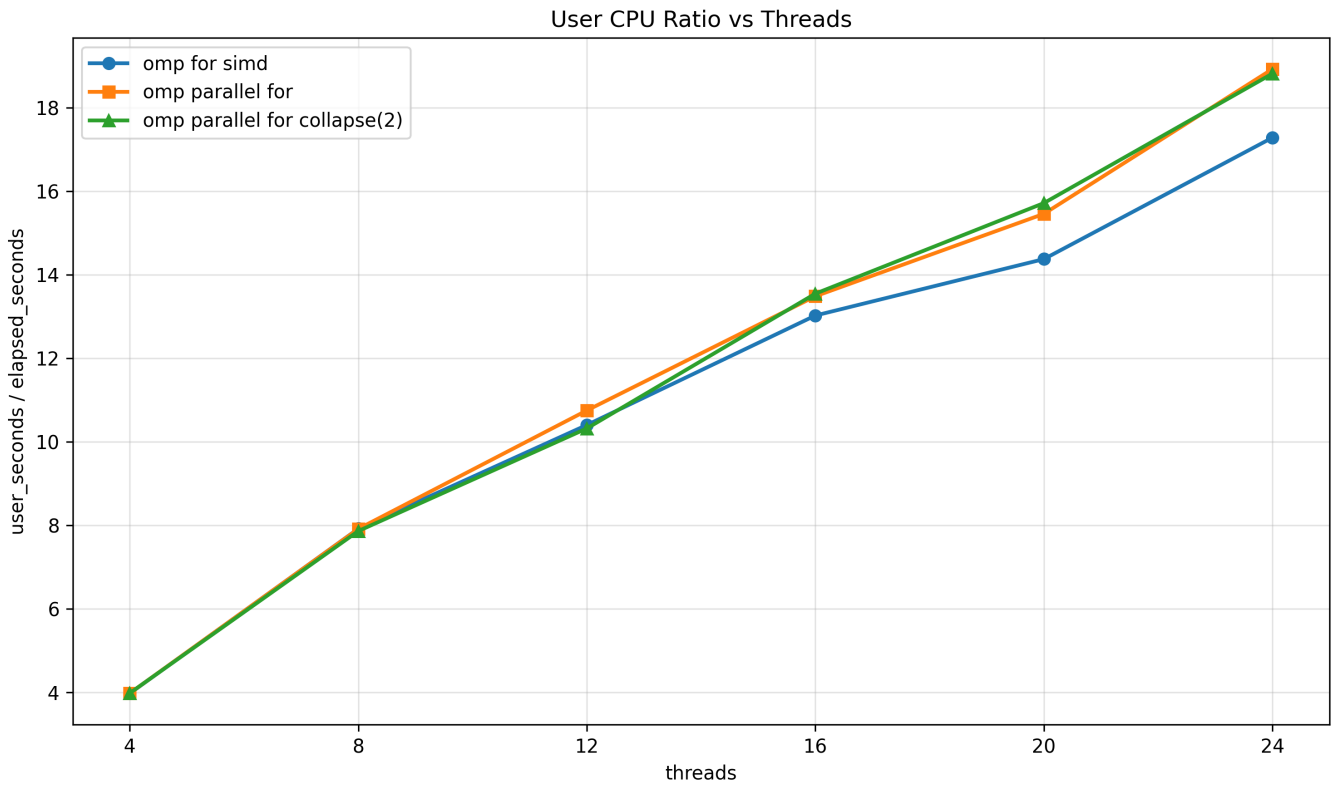

IPC does not increase with thread count and remains relatively stable or even decreases at higher thread counts for all implementations. **omp parallel for** and **omp parallel for collapse(2)** maintain higher IPC values overall, while **omp for simd** stays consistently lower despite achieving the best execution time.

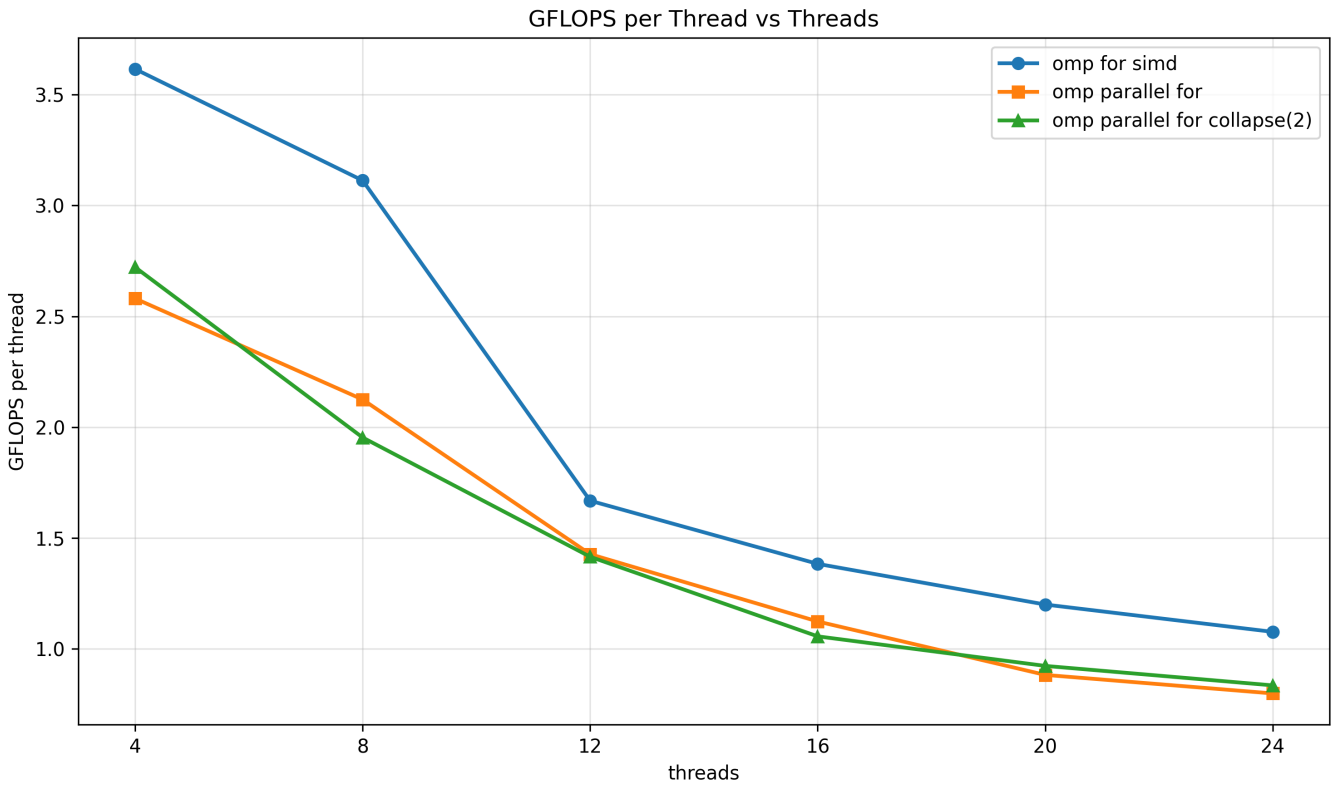
This behavior reflects how efficiently each implementation uses CPU cycles as the number of threads grows. Increasing threads introduces more synchronization, scheduling, and contention for shared resources, which reduces the fraction of cycles spent executing useful instructions. In **omp parallel for**, loop distribution remains efficient, allowing relatively stable IPC, while **collapse(2)** improves load balance but introduces additional scheduling overhead that limits gains at higher thread counts. In **omp for simd**, vectorization improves per-thread computation, but this is not fully reflected in IPC, since IPC measures instruction-level throughput rather than the amount of work done per instruction.

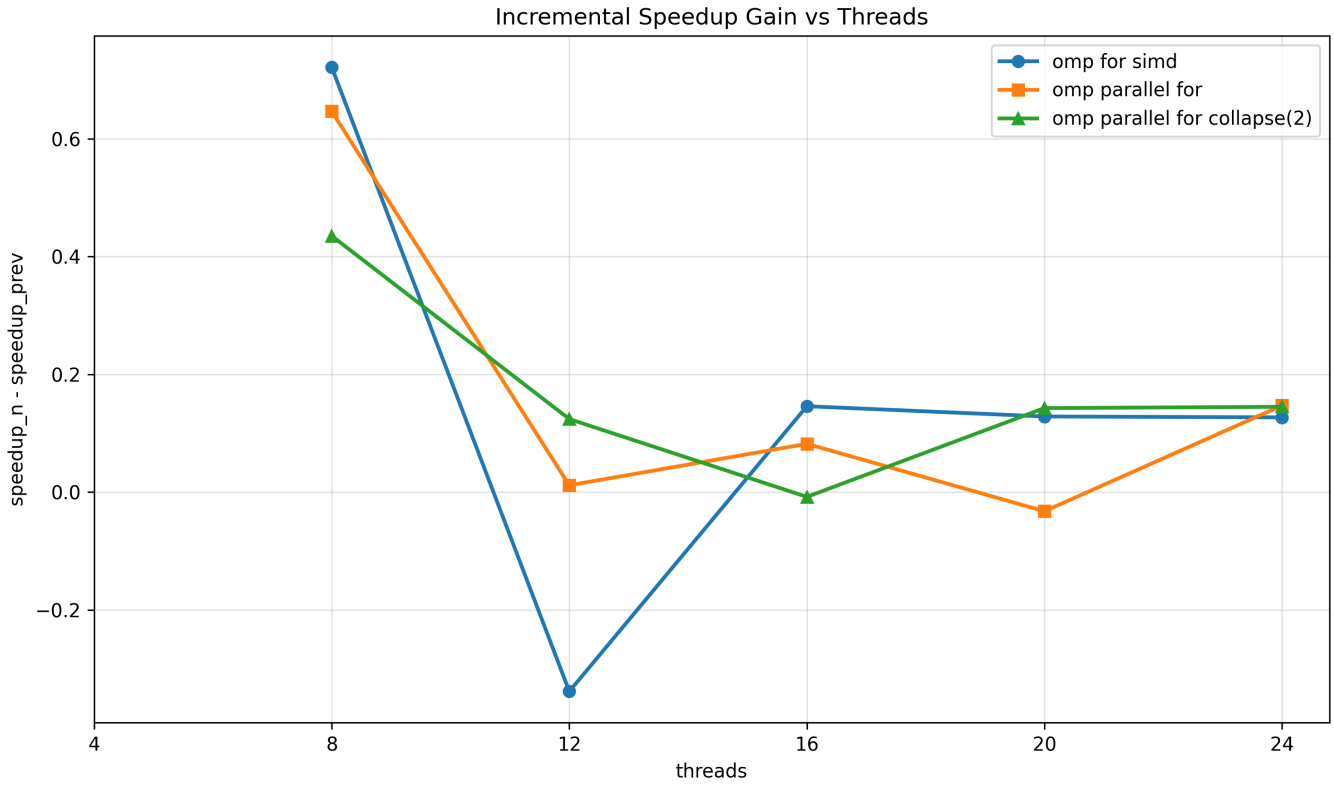
Overall, IPC confirms that adding threads does not necessarily improve per-core efficiency. Even when total performance improves, each core tends to spend a larger fraction of time on coordination and less on useful computation, which explains the saturation observed in speedup and GFLOPS.

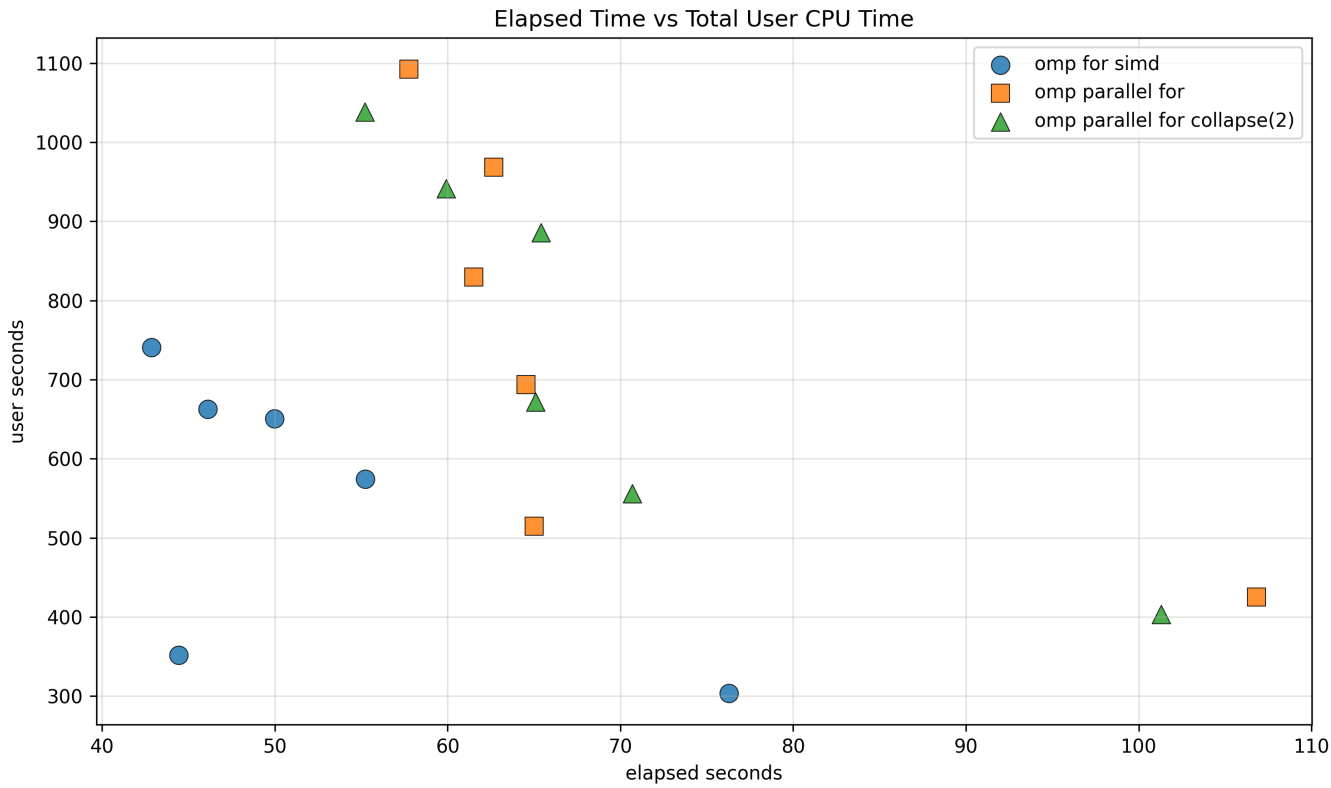
## 5.6 Advanced Scaling Insights




GFLOPS per Thread vs Threads
omp for simd
omp parallel for
omp parallel for collapse(2)
GFLOPS per thread
3.5
3.0
2.5
2.0
1.5
1.0
4
8
12
16
20
24
threads


Incremental Speedup Gain vs Threads
omp for simd
omp parallel for
omp parallel for collapse(2)
speedup_n - speedup_prev
0.6
0.4
0.2
0.0
−0.2
4
8
12
16
20
24
threads

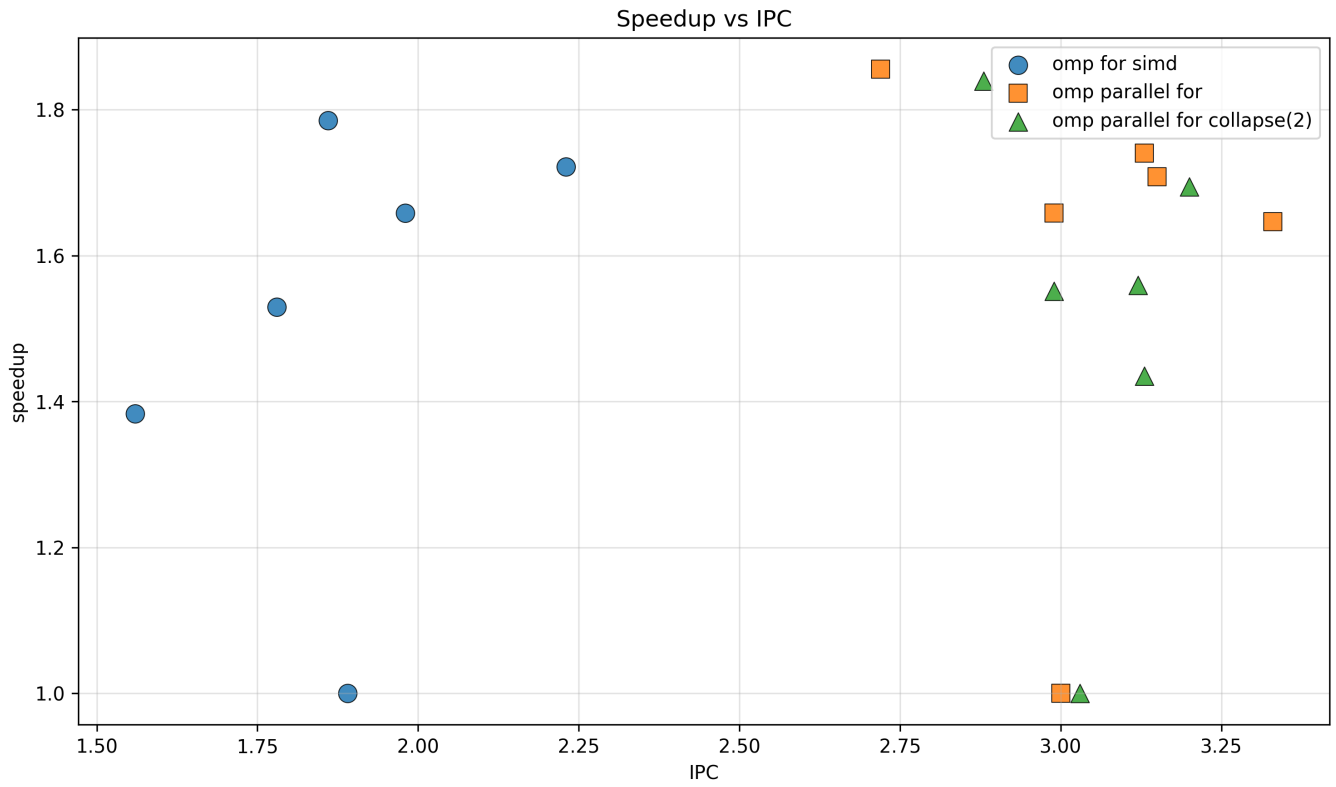



All implementations (**omp parallel for**, **omp parallel for collapse(2)**, and **omp for simd**) show the same fundamental pattern: as the number of threads increases, total CPU work grows significantly while the improvement in elapsed time becomes progressively smaller. This is visible in **user_time_vs_threads** and **user_cpu_ratio_vs_threads**, where higher thread counts require much more aggregate CPU time even when execution time gains are limited.

The incremental metrics make this effect explicit. Both **incremental_speedup_gain_vs_threads** and **incremental_time_reduction_pct_vs_threads** show that each increase in thread count contributes less improvement than the previous one, clearly identifying the point of diminishing returns. This is reinforced by **gflops_per_thread_vs_threads**, where per-thread productivity decreases as more threads are added, indicating that a larger fraction of execution time is spent on synchronization, scheduling, and resource contention rather than useful computation. The **elapsed_vs_user_time_scatter** further highlights this trade-off, showing that reducing elapsed time at higher thread counts requires disproportionately higher CPU effort.

Overall, these results show that the scalability limit is not determined by the maximum number of threads, but by the point where additional threads no longer provide meaningful performance gains due to increasing overhead and contention.

### 5.7 Strategy Comparison




The speedup vs IPC scatter shows that **omp parallel for**, **omp parallel for collapse(2)**, and **omp for simd** follow different trade-offs rather than a single consistent trend. **omp parallel for** and **collapse(2)** tend to achieve higher IPC values, while **omp for simd** often reaches competitive or higher speedup despite lower IPC.

This behavior reflects how each construct balances instruction-level efficiency and thread-level scaling. In **omp parallel for**, low overhead allows both good IPC and solid speedup. **omp parallel for collapse(2)** improves load distribution by increasing the available iteration space, which helps maintain IPC at higher thread counts, but adds scheduling overhead that limits speedup. **omp for simd** increases the amount of work per instruction through vectorization, so even with lower IPC, it can achieve strong speedup because each instruction performs more computation.

Overall, the best-performing approach is not the one with the highest IPC or speedup alone, but the one that maintains a good balance between the two. This highlights that both efficient instruction execution and low parallel overhead are necessary for good scalability.

## 6. Discussion

Exercise 1 shows that **OnMult - omp parallel for** and **OnMultLine - parallel** outperform **OnMult - omp parallel + omp for** because of how the OpenMP directives are applied. The `#pragma omp parallel for` directive creates a team of threads and immediately distributes loop iterations among them, so threads start executing useful work with minimal overhead. In contrast, `#pragma omp parallel` followed by `#pragma omp for` separates thread creation from work distribution, introducing additional synchronization and coordination between these steps. This extra management does not contribute to computation and reduces overall efficiency.

From a threading perspective, OpenMP works by splitting loop iterations across multiple threads that execute concurrently. This is most effective when threads can work independently with little need for

coordination. When the structure introduces extra synchronization points, threads spend more time waiting or managing execution instead of performing computation, which explains the lower performance of **omp parallel + omp for**.

Exercise 2 shows that increasing the number of threads improves performance initially, but only up to a certain point. At low thread counts, each additional thread contributes directly to parallel work, reducing execution time. As the number of threads grows, however, the overhead of managing them increases, and threads begin to compete for shared resources such as CPU execution units and memory bandwidth. This leads to diminishing returns, where adding more threads results in smaller performance gains.

Different directives affect this scaling behavior. **omp parallel for** provides a good balance between low overhead and effective work distribution. **omp parallel for collapse(2)** increases the number of iterations available for distribution by combining nested loops, which can improve load balancing at higher thread counts but also adds scheduling overhead. **omp for simd** improves per-thread performance by using vector instructions, increasing the amount of computation per instruction, but it does not eliminate the overhead associated with coordinating many threads.

Overall, the results show that parallel performance depends on both how threads are created and how work is assigned to them. Threads are most useful when they can execute independently with minimal coordination, and the best OpenMP constructs are those that reduce overhead while keeping threads consistently engaged in useful computation.

## 7. Conclusion

The results from Part 2 show that performance is determined more by how OpenMP directives are used than by the number of threads alone. In Exercise 1, **OnMult - omp parallel for** and **OnMultLine - parallel** outperform **OnMult - omp parallel + omp for** because they minimize synchronization and allow threads to start useful work immediately, while the split structure introduces unnecessary overhead.

In Exercise 2, all implementations benefit from increasing the number of threads, but only up to the point where overhead and resource contention begin to dominate. Beyond this point, additional threads provide diminishing returns, and metrics such as speedup, efficiency, and GFLOPS all reflect this saturation.

Overall, the key takeaway is that effective parallelization requires both low-overhead thread management and balanced work distribution. The best performance is achieved when threads are kept consistently busy with useful computation, and when the chosen OpenMP construct minimizes coordination costs while scaling efficiently with the available hardware.